

Miscellaneous Reflection Cleanup

Document #: P3795R2 [[Latest](#)] [[Status](#)]
Date: 2026-03-26
Project: Programming Language C++
Audience: CWG, LWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Proposal](#)
 - [3.1 Missing Predicates](#)
 - [3.2 Scope Identification](#)
 - [3.3 data_member_spec API](#)
 - [3.4 Annotations on Function Parameters](#)
 - [3.5 Missing Metafunctions](#)
 - [3.6 Inconsistent Error-Handling API](#)
 - [3.7 Specifying Error-Handling More Precisely](#)
- [4 Wording](#)
- [5 References](#)

§ 1 Revision History

Since [\[P3795R1\]](#), specified non-`void` type for function parameter annotations.

Since [\[P3795R0\]](#), fixing missing wording in `annotations_of` and `data_member_spec` (including annotations), rebasing the wording. No longer pursuing the `is_inline`, `is_constexpr`, `is_consteval` functions, those have been removed from this paper.

§ 2 Introduction

At the Sofia meeting, [\[P2996R13\]](#) ([Reflection for C++26](#)), [\[P3394R4\]](#) ([Annotations for Reflection](#)), [\[P3293R3\]](#) ([Splicing a base class subobject](#)), [\[P3491R3\]](#) ([define static {string,object,array}](#)), [\[P3096R12\]](#) ([Function Parameter Reflection in Reflection for C++26](#)), and [\[P3560R2\]](#) ([Error Handling in Reflection](#)) were all adopted. Because these were all (somewhat) independent papers that were adopted at the same time, there were a few inconsistencies that were introduced. Some gaps in coverage. Some inconsistent APIs. This paper just seeks to correct a bunch of those little issues.

§ 3 Proposal

This isn't really one proposal, per se, as much as it is a number of very small proposals.

§ 3.1 Missing Predicates

[\[P3795R0\]](#) proposed adding the predicates `is_inline(r)`, `is_constexpr(r)`, and `is_consteval(r)`. After LEWG discussion in Kona, with questions about what `is_constexpr(r)` actually means and whether it applies to implicitly `constexpr` functions as well as implicitly-declared ones, this proposal is simply no longer pursuing those.

§ 3.2 Scope Identification

[\[P2996R13\]](#)'s mechanism for access checking relies on `std::meta::access_context`. However, that facility exposes another interesting bit of functionality — albeit in a very indirect way. You can identify what class you're currently in:

```
consteval auto current_class(std::meta::info scope = std::meta::access_
    context::current().scope())
-> std::meta::info
{
    while (true) {
        if (is_type(scope)) {
            return scope;
        }

        if (is_namespace(scope)) {
            throw std::meta::exception(/* ... */);
        }

        scope = parent_of(scope);
    }
}
```

Given that this is a useful (and occasionally asked for) piece of information, we should just provide it directly, rather than having this API proliferate.

The same is true for `current_function` and `current_namespace`. The only open question in my mind is whether there are situations in which you'd want something like a `nearest_enclosing_class_or_namespace`.

§ 3.3 `data_member_spec` API

The current API for `std::meta::define_aggregate` requires calls to `std::meta::data_member_spec`, which currently looks like this:

```
constexpr info data_member_spec(info type, data_member_options options);
```

When we originally proposed this API, we allowed the *name* of a non-static data member to be omitted. Doing so was akin to asking the implementation to create a unique name for you. However, that changed in [\[P2996R8\]](#) such that if `options.name` were not provided then the data member had to be an unnamed bit-field (which meant that `options.width` had to be provided). That means that while we originally envisioned it being possible to implement tuple like this:

```
constexpr {
    define_aggregate(
        ^^storage,
        {data_member_spec(^^Ts)...}
    );
}
```

That's no longer actually possible. You always have to provide *some* member of options. So tuple looks more like this (note that you're allowed to have multiple members named `_now`):

```
constexpr {
    define_aggregate(
        ^^storage,
        {data_member_spec(^^Ts, {.name="_"})...}
    );
}
```

As such, it looks a little odd to have the type off by itself like that. It's true that you *always* need to provide a type, but why is it special? Let's face it, approximately nobody is going to be creating unnamed bit-fields, so the name is practically always present too.

A previous revision of this paper suggested making this change to the API:

```
struct data_member_options {
    struct name-type { // exposition only
        // ...
    };

+   info type;
    optional<name-type> name;
    optional<int> alignment;
    optional<int> bit_width;
    bool no_unique_address = false;
};

-   constexpr info data_member_spec(info type, data_member_options
        options);
+   constexpr info data_member_spec(data_member_options options);
```

However, Jakub Jelinek pointed out to me during the Croydon meeting that by moving the type option within `data_member_options`, we no longer have a top-level argument of reflection type, and thus argument-dependent lookup doesn't kick in. As a result, the impact of the above change for typical uses would be:

```
- data_member_spec(type, {.name=name})  
+ std::meta::data_member_spec({.type=type, .name=name})
```

We thought the benefits of keeping the options altogether wasn't as high as the benefit of not having to qualify the call, so this revision doesn't make a change in this area.

However, while adding *attributes* is a difficult question, adding *annotations* is actually much simpler. With the adoption of [\[P3394R4\]](#), we definitely should allow you to add annotations to your generated members. The proposed change as a result (partially reverted from [\[P3795R1\]](#)) is:

```
struct data_member_options {  
    struct name_type { // exposition only  
        // ...  
    };  
  
    optional<name_type> name;  
    optional<int> alignment;  
    optional<int> bit_width;  
    bool no_unique_address = false;  
+    vector<info> annotations = {};  
};  
  
constexpr info data_member_spec(info type, data_member_options options);
```

§ 3.4 Annotations on Function Parameters

[\[P3394R4\]](#) ([Annotations for Reflection](#)) and [\[P3096R12\]](#) ([Function Parameter Reflection in Reflection for C++26](#)) were independent proposals adopted at the same time. The former gave us annotations, but the latter gave us the ability to introspect function parameters. Neither could really, directly add support for adding annotations onto function parameters. So, as a result, we don't have them.

But there isn't any particular reason why we shouldn't support this:

```
auto f([ [=1] ] int x) -> void;  
  
constexpr auto a = annotations_of(  
    parameters_of(^f)[0]  
)[0];  
static_assert([: constant_of(a) :] == 1);
```

§ 3.5 Missing Metafunctions

[P1317R2] [\(Remove return type deduction in std::apply\)](#) was also adopted in Sofia. It added three new type traits, but none of us were aware of that paper, much less expected it to be adopted, so we neglected to add consteval metafunction equivalents to those three type traits:

```
consteval bool is_applicable_type(info fn, info tuple);
consteval bool is_nothrow_applicable_type(info fn, info tuple);
consteval info apply_result(info fn, info tuple);
```

§ 3.6 Inconsistent Error-Handling API

This section in [P3795R0] pointed out how some functions in `std::meta::` still had a *Constant When* specification instead of a *Throws* specification, but that was already resolved directly by the handling of [P3560R2]. Nothing more is needed here.

§ 3.7 Specifying Error-Handling More Precisely

Currently, [P3560R2] specifies nothing about the contents of the exceptions being thrown on failure from various reflection functions. That's largely as it should be — we really don't need to specify the message, for instance.

But one thing that `std::meta::exception` gives you is a `from()` accessor that tells you which operation failed. And that we *should* specify — providing some front-matter sentence that says that when a function or function template `F` in `<meta>` is specified to throw a `meta::exception`, that `from()` is `^^F`.

§ 4 Wording

Extend what an annotation can represent in 9.13.12 [\[dcl.attr.annotation\]](#):

- 1 An annotation may be applied to any declaration of a type, type alias, variable, function, **function parameter of non-void type**, namespace, enumerator, *base-specifier*, or non-static data member.

[*Note* 1*: An annotation on a *parameter-declaration* in a function definition applies to both the function parameter and the variable.

[*Example 1*:

```
void f([1] int x);
void f([2] int y) {
    constexpr auto rp = parameters_of(f)[0];
    constexpr auto ry = variable_of(rp);
    static_assert(ry == y);

    static_assert(annotations_of(rp).size() == 2); // both [1, 2]
    static_assert(annotations_of(ry).size() == 1); // just [2]
```

```

    static_assert(annotations_of(rp)[1] == annotations_of(ry)[0]);
}

— end example ]
— end note ]

```

Change 11.4.1 [\[class.mem.general\]](#) to extend our quintuple to a sextuple:

- 32 A data member description is a **quintuple sextuple** (T, N, A, W, NUA, ANN) describing the potential declaration of a non-static data member where
- (32.1) — T is a type,
 - (32.2) — N is an *identifier* or $\perp$,
 - (32.3) — A is an alignment or $\perp$,
 - (32.4) — W is a bit-field width or $\perp$, **and**
 - (32.5) — NUA is a boolean value, **and**
 - (32.6) — **ANN is a sequence of reflections representing either values or template parameter objects.**

Two data member descriptions are equal if each of their respective components are the same **entities** **entity**, **are** the same identifiers, **have equal values** **the same value**, the same **sequence**, or **are** both $\perp$.

The synopsis change for 21.4.1 [\[meta.syn\]](#) is:

```

#include <initializer_list>

namespace std::meta {
    using info = decltype(^^::);

    // ...

    // [meta.reflection.access.context], access control context
    struct access_context;

    // [meta.reflection.access.queries], member accessability queries
    consteval bool is_accessible(info r, access_context ctx);
    consteval bool has_inaccessible_nonstatic_data_members(info r, access_context ctx);
    consteval bool has_inaccessible_bases(info r, access_context ctx);
    consteval bool has_inaccessible_subobjects(info r, access_context ctx);

    + // [meta.reflection.scope], scope identification
    + consteval info current_function();
    + consteval info current_class();
    + consteval info current_namespace();

```

```

// [meta.reflection.member.queries], reflection member queries
// ...

// [meta.reflection.define.aggregate], class definition generation
struct data_member_options;
    consteval info data_member_spec(info type, data_member_options
        options);
consteval bool is_data_member_spec(info r);
template<reflection_range R = initializer_list<info>>
    consteval info define_aggregate(info type_class, R&&);

// associated with [meta.unary.cat], primary type categories
// ...

// associated with [meta.trans.other], other transformations
consteval info remove_cvref(info type);
consteval info decay(info type);
template<reflection_range R = initializer_list<info>>
    consteval info common_type(R&& type_args);
template<reflection_range R = initializer_list<info>>
    consteval info common_reference(R&& type_args);
consteval info type_underlying_type(info type);
template<reflection_range R = initializer_list<info>>
    consteval info invoke_result(info type, R&& type_args);
consteval info unwrap_reference(info type);
consteval info unwrap_ref_decay(info type);

consteval size_t tuple_size(info type);
consteval info tuple_element(size_t index, info type);
+ consteval bool is_applicable_type(info fn, info tuple);
+ consteval bool is_nothrow_applicable_type(info fn, info tuple);
+ consteval info apply_result(info fn, info tuple);

consteval size_t variant_size(info type);
consteval info variant_alternative(size_t index, info type);

consteval strong_ordering type_order(info type_a, info type_b);

// [meta.reflection.annotation], annotation reflection
consteval vector<info> annotations_of(info item);
consteval vector<info> annotations_of_with_type(info item, info type);
}

```

Add to the front matter in 21.4.1 [\[meta.syn\]](#):

- 1 Unless otherwise specified, each function, and each specialization of any function template, specified in this header is a designated addressable function ([namespace.std]).

* When a function or function template specialization *F* specified in this header throws a `meta::exception` *E*, *E*.`from()` is a reflection representing *F* and *E*.`where()` is a `source_location` representing from where the call to *F* originated.

- 2 The behavior of any function specified in namespace `std::meta` is implementation-defined when a reflection of a construct not otherwise specified by this document is provided as an argument.

Adjust the data member description wording in 21.4.6 [\[meta.reflection.names\]](#):

- ```
constexpr bool has_identifier(info r);
```
- 1 *Returns* [...]
- (1.1) — [...]
- (1.13) — Otherwise, *r* represents a data member description ( $T, N, A, W, NUA, ANN$ ) ([class.mem.-general]); `true` if *N* is not  $\perp$ . Otherwise, `false`.
- ```
constexpr string_view identifier_of(info r);
constexpr u8string_view u8identifier_of(info r);
```
- 2 Let *E* be [...]
- 3 *Returns*: An NTMBS, encoded with *E*, determined as follows:
- (3.1) — [...]
- (3.6) — Otherwise, *r* represents a data member description (T, N, A, W, NUA, ANN) ([class.mem.-general]); a `string_view` or `u8string_view`, respectively, containing the identifier *N*.

Adjust the data member description wording in 21.4.7 [\[meta.reflection.queries\]](#):

- ```
constexpr info type_of(info r);
```
- 2 *Returns*:
- (2.6) — Otherwise, for a data member description ( $T, N, A, W, NUA, ANN$ ) ([class.mem.general]), a reflection of the type *T*.
- ```
constexpr bool is_bit_field(info r);
```
- 19 *Returns*: `true` if *r* represents a bit-field, or if *r* represents a data member description (T, N, A, W, NUA, ANN) ([class.mem.general]) for which *W* is not $\perp$. Otherwise, `false`.

Add the new subclause [\[meta.reflection.scope\]](#) before 21.4.8 [\[meta.reflection.access.context\]](#). [Drafting note: The wording for *eval-point* (p3) and *ctx-scope* (p4) is moved wholesale from `access_context::current`. Paragraphs 1, 2, and 5 onwards are new]:

- 1 The functions in this subclause retrieve information about where in the program they are invoked.

2 None of the functions in this subclause is an addressable function ([namespace.std]).

3 Given a program point P , let $eval\text{-}point(P)$ be the following program point:

(3.1) — If a potentially-evaluated subexpression ([intro.execution]) of a default member initializer I for a member of a class C ([class.mem.general]) appears at P , then a point determined as follows:

(3.1.1) — If an aggregate initialization is using I , $eval\text{-}point(Q)$, where Q is the point at which that aggregate initialization appears.

(3.1.2) — Otherwise, if an initialization by an inherited constructor ([class.inhctor.init]) is using I , a point whose immediate scope is the class scope corresponding to C .

(3.1.3) — Otherwise, a point whose immediate scope is the function parameter scope corresponding to the constructor definition that is using I .

(3.2) — Otherwise, if a potentially-evaluated subexpression of a default argument ([dcl.fct.default]) appears at P , $eval\text{-}point(Q)$, where Q is the point at which the invocation of the function ([expr.call]) using that default argument appears.

(3.3) — Otherwise, if the immediate scope of P is a function parameter scope introduced by a declaration D , and P appears either before the locus of D or within the trailing *requires-clause* of D , a point whose immediate scope is the innermost scope enclosing the locus of D that is not a template parameter scope.

(3.4) — Otherwise, if the immediate scope of P is a function parameter scope introduced by a *lambda-expression* L whose *lambda-introducer* appears at point Q , and P appears either within the *trailing-return-type* or the trailing *requires-clause* of L , $eval\text{-}point(Q)$.

(3.5) — Otherwise, if the innermost non-block scope enclosing P is the function parameter scope introduced by a *constexpr-block-declaration* ([dcl.pre]), a point whose immediate scope is that inhabited by the outermost *constexpr-block-declaration* D containing P such that each scope (if any) that intervenes between P and the function parameter scope introduced by D is either

(3.5.1) — a block scope or

(3.5.2) — a function parameter scope or lambda scope introduced by a *constexpr-block-declaration*.

(3.6) — Otherwise, P .

4 Given a scope S , let $ctx\text{-}scope(S)$ be the following scope:

(4.1) — If S is a class scope or a namespace scope, S .

(4.2) — Otherwise, if S is a function parameter scope introduced by the declaration of a function, S .

(4.3) — Otherwise, if S is a lambda scope introduced by a *lambda-expression* L , the function parameter scope corresponding to the call operator of the closure type for L .

(4.4) — Otherwise, `ctx-scope(S')` where `S'` is the parent scope of `S`.

5 Let `CURRENT-SCOPE(P)` for a point `P` be a reflection representing the function, class, or namespace whose corresponding function parameter scope, class scope, or namespace scope, respectively, is `ctx-scope(S)`, where `S` is the immediate scope of `eval-point(P)`.

```
consteval info current_function();
```

6 An invocation of `current_function` that appears at a program point `P` is value-dependent ([temp.dep.ctxexpr]) if `eval-point(P)` is enclosed by a scope corresponding to a templated entity.

7 Let `S` be `CURRENT-SCOPE(P)` where `P` is the point at which the invocation of `current_function` lexically appears.

8 *Throws:* `meta::exception` unless `S` represents a function.

9 *Returns:* `S`.

```
consteval info current_class();
```

10 An invocation of `current_class` that appears at a program point `P` is value-dependent ([temp.dep.ctxexpr]) if `eval-point(P)` is enclosed by a scope corresponding to a templated entity.

11 Let `S` be `CURRENT-SCOPE(P)` where `P` is the point at which the invocation of `current_class` lexically appears.

12 *Throws:* `meta::exception` unless `S` represents either a class or a member function.

13 *Returns:* `S` if `S` represents a class. Otherwise, `parent_of(S)`.

```
consteval info current_namespace();
```

14 An invocation of `current_namespace` that appears at a program point `P` is value-dependent ([temp.dep.ctxexpr]) if `eval-point(P)` is enclosed by a scope corresponding to a templated entity.

15 Let `S` be `CURRENT-SCOPE(P)` where `P` is the point at which the invocation of `current_namespace` lexically appears.

16 *Returns:* `S` if `S` represents a namespace. Otherwise, a reflection representing the nearest enclosing namespace of the entity represented by `S`.

Adjust down the now-moved wording from 21.4.8 [[meta.reflection.access.context](#)]:

```
static consteval access_context current() noexcept;
```

5 `current` is not an addressable function ([namespace.std]).

6

Given a program point P , let ~~$\text{eval_point}(P)$~~ be the following program point:

- (6.1) — If a potentially-evaluated subexpression (~~$\{\text{intro.execution}\}$~~) of a default member initializer I for a member of a class C (~~$\{\text{class.mem.general}\}$~~) appears at P , then a point determined as follows:
 - (6.1.1) — If an aggregate initialization is using I , ~~$\text{eval_point}(Q)$~~ , where Q is the point at which that aggregate initialization appears.
 - (6.1.2) — Otherwise, if an initialization by an inherited constructor (~~$\{\text{class.inhctor.init}\}$~~) is using I , a point whose immediate scope is the class scope corresponding to C .
 - (6.1.3) — Otherwise, a point whose immediate scope is the function parameter scope corresponding to the constructor definition that is using I .
- (6.2) — Otherwise, if a potentially-evaluated subexpression of a default argument (~~$\{\text{def.fct.default}\}$~~) appears at P , ~~$\text{eval_point}(Q)$~~ , where Q is the point at which the invocation of the function (~~$\{\text{expr.call}\}$~~) using that default argument appears.
- (6.3) — Otherwise, if the immediate scope of P is a function parameter scope introduced by a declaration D , and P appears either before the locus of D or within the trailing ~~requires-clause~~ of D , a point whose immediate scope is the innermost scope enclosing the locus of D that is not a template parameter scope.
- (6.4) — Otherwise, if the immediate scope of P is a function parameter scope introduced by a ~~lambda-expression~~ L whose ~~lambda-introducer~~ appears at point Q , and P appears either within the ~~$\text{trailing-return-type}$~~ or the trailing ~~$\text{requires-clause}$~~ of L , ~~$\text{eval_point}(Q)$~~ .
- (6.5) — Otherwise, if the innermost non-block scope enclosing P is the function parameter scope introduced by a ~~$\text{consteval-block-declaration}$~~ (~~$\{\text{del.pre}\}$~~), a point whose immediate scope is that inhabited by the outermost ~~$\text{consteval-block-declaration}$~~ D containing P such that each scope (if any) that intervenes between P and the function parameter scope introduced by D is either
 - (6.5.1) — a block scope or
 - (6.5.2) — a function parameter scope or lambda scope introduced by a ~~$\text{consteval-block-declaration}$~~ .
- (6.6) — Otherwise, P .

7

Given a scope S , let ~~$\text{ctx_scope}(S)$~~ be the following scope:

- (7.1) — If S is a class scope or a namespace scope, S .
- (7.2) — Otherwise, if S is a function parameter scope introduced by the declaration of a function, S .
- (7.3) — Otherwise, if S is a lambda scope introduced by a ~~lambda-expression~~ L , the function parameter scope corresponding to the call operator of the closure type for L .
- (7.4) — Otherwise, ~~$\text{ctx_scope}(S')$~~ where S' is the parent scope of S .

- 8 An invocation of `current` that appears at a program point P is value-dependent ([temp.dep.ctxpr]) if `eval-point( $P$ )` is enclosed by a scope corresponding to a templated entity.
- 9 *Returns:* An `access_context` whose designating class is the null reflection and whose scope ~~represents the function, class, or namespace whose corresponding function parameter scope, class scope, or namespace scope is `ctx-scope( $S$ )`, where S is the immediate scope of `eval-point( $P$ )` and~~ is `CURRENT-SCOPE( $P$ )` where P is the point at which the invocation of `current` lexically appears.

Adjust the data member description wording in 21.4.11 [\[meta.reflection.layout\]](#):

- ```
constexpr size_t size_of(info r);
```
- 5 *Returns:*
- (5.1) — If  $r$  represents a non-static data member of type  $T$  or a data member description  $(T, N, A, W, NUA, ANN)$ , or
- 6 *Throws:* `meta::exception` unless all of the following conditions are met:
- (6.1) — `dealias( $r$ )` is a reflection of a type, object, value, variable of non-reference type, non-static data member that is not a bit-field, direct base class relationship, or data member description  $(T, N, A, W, NUA, ANN)$  ([class.mem.general]) where  $W$  is  $\perp$ .
- ```
constexpr size_t alignment_of(info r);
```
- 7 *Returns:*
- (7.5) — Otherwise, r represents a data member description (T, N, A, W, NUA, ANN) ([class.mem.-general]). If A is not $\perp$, then the value A . Otherwise, `alignment_of( $\wedge T$ )`.
- 8 *Throws:* `meta::exception` unless all of the following conditions are met:
- (8.1) — `dealias( $r$ )` is a reflection of a type, object, variable of non-reference type, non-static data member that is not a bit-field, direct base class relationship, or data member description (T, N, A, W, NUA, ANN) ([class.mem.general]) where W is $\perp$.
- ```
constexpr size_t bit_size_of(info r);
```
- 9 *Returns:*
- (9.2) — Otherwise, if  $r$  represents a data member description  $(T, N, A, W, NUA, ANN)$  and  $W$  is not  $\perp$ , then  $W$ .

Change 21.4.12 [\[meta.reflection.annotation\]](#):

[ Drafting note: This doesn't work for data member specifications, because those contain constants — not annotations yet. Is it even useful to pull them back out? They're not annotations yet. Do we have to synthesize annotations? It's useful to add annotations to generated data members — I'm not sure if it's useful

to query the annotations on a pre-generated data member? There's also a drive-by cleanup of the wording around direct base class relationships, since *base-specifiers* aren't technically declared. ]

```
constexpr vector<info> annotations_of(info item);
```

1 For a function  $F$ , let  $S(F)$  be the set of declarations, ignoring any explicit instantiations, that declare either  $F$  or a templated function of which  $F$  is a specialization.

1 ~~Let  $E$  be~~

(1.1) ~~— the corresponding *base-specifier* if  $item$  represents a direct base class relationship,~~

(1.2) ~~— otherwise, the entity represented by  $item$ .~~

2 *Returns*: a vector containing all of the reflections  $R$  representing each annotation applying to ~~each declaration of  $E$  that~~:

(2.1) — if  $item$  represents a function parameter  $P$  of a function  $F$ , then the declaration of  $P$  in each declaration of  $F$  in  $S(F)$ ,

(2.2) — otherwise, if  $item$  represents a function  $F$ , then each declaration of  $F$  in  $S(F)$ ,

(2.3) — otherwise, if  $item$  represents a direct base class relationship  $(D, B)$ , then the corresponding *base-specifier* in the definition of  $D$ ,

(2.4) — otherwise, each declaration of the entity represented by  $item$ ,

such that each specified declaration precedes either some point in the evaluation context (`[expr.const]`) or a point immediately following the *class-specifier* of the outermost class for which such a point is in a complete-class context. For any two reflections  $R_1$  and  $R_2$  in the returned vector, if the annotation represented by  $R_1$  precedes the annotation represented by  $R_2$ , then  $R_1$  appears before  $R_2$ . If  $R_1$  and  $R_2$  represent annotations from the same translation unit  $T$ , any element in the returned vector between  $R_1$  and  $R_2$  represents an annotation from  $T$ .

[ *Note 2*: The order in which two annotations appear is otherwise unspecified. — *end note* ]

3 *Throws*: `meta::exception` unless  $item$  represents a type, type alias, variable, function, **function parameter**, namespace, enumerator, direct base class relationship, or non-static data member.

Change the `data_member_spec` API in 21.4.16 [\[meta.reflection.define.aggregate\]](#) [ *Drafting note*: The nested example in the note is now separate from the note ]:

```
struct data_member_options {
 struct name_type { // exposition only
 template<class T> requires constructible_from<u8string, T>
```

```

 consteval name-type(T &&);

 template<class T> requires constructible_from<string, T>
 consteval name-type(T &&);

private:
 variant<u8string, string> contents; // exposition only
};

optional<name-type> name;
optional<int> alignment;
optional<int> bit_width;
bool no_unique_address = false;
+ vector<info> annotations;
};

```

- 1 The classes `data_member_options` and `data_member_options::name-type` are consteval-only types ([basic.types.general]), and are not structural types ([temp.param]).

```

template <class T> requires constructible_from<u8string, T>
consteval name-type(T&& value);

```

- 2 *Effects*: Initializes `contents` with `u8string(std::forward<T>(value))`.

```

template<class T> requires constructible_from<string, T>
consteval name-type(T&& value);

```

- 3 *Effects*: Initializes `contents` with `string(std::forward<T>(value))`.

[ *Note 3*: The class `name-type` allows the function `data_member_spec` to accept an ordinary string literal (or `string_view`, `string`, etc.) or a UTF-8 string literal (or `u8string_view`, `u8string`, etc.) equally well. — *end note* ]

[ *Example 2*:

```

consteval void fn() {
 data_member_options o1 = {.name="ordinary_literal_encoding"};
 data_member_options o2 = {.name=u8"utf8_encoding"};
}

```

— *end example* ]

```

consteval info data_member_spec(info type,
 data_member_options options);

```

- 4 *Returns*: A reflection of a data member description ( $T, N, A, W, NUA, ANN$ ) ([class.mem.general]) where

- (4.1) —  $T$  is the type represented by `dealias(type)`,
- (4.2) —  $N$  is either the identifier encoded by `options.name` or  $\perp$  if `options.name` does not contain a value,

- (4.3) —  $A$  is either the alignment value held by `options.alignment` or  $\perp$  if `options.alignment` does not contain a value,
- (4.4) —  $W$  is either the value held by `options.bit_width` or  $\perp$  if `options.bit_width` does not contain a value, **and**
- (4.5) —  $NUA$  is the value held by `options.no_unique_address`, **and**
- (4.6) —  **$ANN$  is the sequence of values `constant_of(r)` for each  $r$  in `options.annotations`.**

[ *Note 4*: The returned reflection value is primarily useful in conjunction with `define_aggregate`; it can also be queried by certain other functions in `std::meta` (e.g., `type_of`, `identifier_of`). — *end note* ]

5 *Throws*: `meta::exception` unless the following conditions are met:

- (5.1) — `dealias(type)` represents either an object type or a reference type;
- (5.2) — if `options.name` contains a value, then:
  - (5.2.1) — `holds_alternative<u8string>(options.name->contents)` is `true` and `get<u8string>(options.name->contents)` contains a valid identifier ([`lex.name`]) that is not a keyword ([`lex.key`]) when interpreted with UTF-8, or
  - (5.2.2) — `holds_alternative<string>(options.name->contents)` is `true` and `get<string>(options.name->contents)` contains a valid identifier that is not a keyword when interpreted with the ordinary literal encoding;

[ *Note 5*: The name corresponds to the spelling of an identifier token after phase 6 of translation ([`lex.phases`]). Lexical constructs like *universal-character-names* [`lex.universal.char`] are not processed and will cause evaluation to fail. For example, `R"(\u03B1)"` is an invalid identifier and is not interpreted as `"α"`. — *end note* ]

- (5.3) — if `options.name` does not contain a value, then `options.bit_width` contains a value **and `options.annotations` is empty**;
- (5.4) — if `options.bit_width` contains a value  $V$ , then
  - (5.4.1) — `is_integral_type(type) || is_enumeration_type(type)` is `true`,
  - (5.4.2) — `options.alignment` does not contain a value,
  - (5.4.3) — `options.no_unique_address` is `false`, and
  - (5.4.4) — if  $V$  equals `0` then `options.name` does not contain a value; **and**
- (5.5) — if `options.alignment` contains a value, it is an alignment value ([`basic.align`]) not less than `alignment_of(type)`; **and**
- (5.6) — **for every reflection  $r$  in `options.annotations`, `has-type(r)` is `true`, `type_of(r)` represents a non-array object type, and evaluation of `constant_of(r)` does not exit via an exception.**

```
template<reflection_range R = initializer_list<info>>
constexpr info define_aggregate(info class_type, R&& mdescrs);
```

7 Let  $C$  be the class represented by `class_type` and  $r_K$  be the  $K^{\text{th}}$  reflection value in `mdescrs`. For every  $r_K$  in `mdescrs`, let  $(T_K, N_K, A_K, W_K, NUA_K, ANN_K)$  be the corresponding data member description represented by  $r_K$ .

8 *Constant When:* [...]

9 Produces an injected declaration  $D$  (`[expr.const]`) that defines  $C$  and has properties as follows:

(9.1) — The target scope of  $D$  is [...]

(9.2) — The locus of  $D$  [...]

(9.3) — The characteristic sequence of  $D$  [...]

(9.4) — If  $C$  is a specialization [...]

(9.5) — For each  $r_K$ , there is a corresponding entity  $M_K$  belonging to the class scope of  $D$  with the following properties:

(9.5.1) — If  $N_K$  is  $\perp$ ,  $M_K$  is an unnamed bit-field. Otherwise,  $M_K$  is a non-static data member whose name is the identifier determined by the character sequence encoded by  $N_K$  in UTF-8.

(9.5.2) — The type of  $M_K$  is  $T_K$ .

(9.5.3) —  $M_K$  is declared with the attribute `[[no_unique_address]]` if and only if  $NUA_K$  is `true`.

(9.5.4) — If  $W_K$  is not  $\perp$ ,  $M_K$  is a bit-field whose width is that value. Otherwise,  $M_K$  is not a bit-field.

(9.5.5) — If  $A_K$  is not  $\perp$ ,  $M_K$  has the *alignment-specifier* `alignas( $A_K$ )`. Otherwise,  $M_K$  has no *alignment-specifier*.

(9.5.6) — For every reflection  $r$  in  $ANN_K$ ,  $M_K$  has an annotation whose underlying constant (`[dcl.attr.annotation]`) is  $r$ .

(9.6) — For every  $r_L$  in `mdescrs` such that  $K < L$  [...]

## § 5 References

[P1317R2] Aaryaman Sagar and Eric Niebler. 2025-06-19. Remove return type deduction in `std::apply`. <https://wg21.link/p1317r2>

[P2996R13] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. 2025-06-18. Reflection for C++26. <https://wg21.link/p2996r13>



[P2996R8] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-12-17. Reflection for C++26.

<https://wg21.link/p2996r8>

[P3096R12] Adam Lach, Dan Katz, Walter Genovese. 2025-06-20. Function Parameter Reflection in Reflection for C++26.

<https://wg21.link/p3096r12>

[P3293R3] Peter Dimov, Dan Katz, Barry Revzin, and Daveed Vandevoorde. 2025-06-19. Splicing a base class subobject.

<https://wg21.link/p3293r3>

[P3394R4] Wyatt Childers, Dan Katz, Barry Revzin, and Daveed Vandevoorde. 2025-06-09. Annotations for Reflection.

<https://wg21.link/p3394r4>

[P3491R3] Wyatt Childers, Peter Dimov, Barry Revzin, and Daveed Vandevoorde. 2025-06-20. `define_static_{string,object,array}`.

<https://wg21.link/p3491r3>

[P3560R2] Peter Dimov and Barry Revzin. 2025-06-17. Error Handling in Reflection.

<https://wg21.link/p3560r2>

[P3795R0] Barry Revzin. 2025-07-15. Miscellaneous Reflection Cleanup.

<https://wg21.link/p3795r0>

[P3795R1] Barry Revzin. 2026-01-09. Miscellaneous Reflection Cleanup.

<https://wg21.link/p3795r1>